Abhay Kiran M

261100620002

CO1

1. **LED blink**

```
#include <stdint.h>
#include "stm32f103_regs.h"

static volatile uint32_t g_msticks;

void SysTick_Handler(void)
{
   g_msticks++;
}

static void delay_ms(uint32_t ms)
{
   uint32_t target = g_msticks + ms;
   while (g_msticks < target) {
   }
}

int main(void)
{
   /* enable GPIOC clock */
   RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;

   /* PC13: output push-pull, 2 MHz (pin 13 -> CRH, nibble at bit 20) */
   GPIOC->CRH &= ~(0xFU << 20);
   GPIOC->CRH |=  (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << 20);

   /* SysTick at the default HSI 8 MHz core clock -> 1 kHz tick */
   SYSTICK->LOAD = (8000000U / 1000U) - 1U;
   SYSTICK->VAL  = 0;
   SYSTICK->CTRL = SYSTICK_CTRL_CLKSOURCE | SYSTICK_CTRL_TICKINT |
SYSTICK_CTRL_ENABLE;

   for (;;) {
      GPIOC->ODR ^= (1U << 13);
      delay_ms(500);
   }
}
```

2. **Three thread Blink**

```
#include <stdint.h>
#include "stm32f103_regs.h"

#include "FreeRTOS.h"
```

```
#include "task.h"

typedef struct {
   GPIO_TypeDef *port;
   uint32_t      pin;
   uint32_t      period_ms;
} led_task_args_t;

static const led_task_args_t led_a = { GPIOC, 13, 1000 };
static const led_task_args_t led_b = { GPIOA, 0,  2000 };
static const led_task_args_t led_c = { GPIOA, 1,  3000 };

static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
   volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
   uint32_t shift = (pin % 8U) * 4U;

   *cr &= ~(0xFU << shift);
   *cr |=  (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

static void led_task(void *pvParameters)
{
   const led_task_args_t *args = (const led_task_args_t *)pvParameters;

   for (;;) {
      args->port->ODR ^= (1U << args->pin);
      vTaskDelay(pdMS_TO_TICKS(args->period_ms));
   }
}

int main(void)
{
   RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPCEN;

   gpio_output_init(led_a.port, led_a.pin);
   gpio_output_init(led_b.port, led_b.pin);
   gpio_output_init(led_c.port, led_c.pin);

   xTaskCreate(led_task,  "led_1s",  configMINIMAL_STACK_SIZE,  (void  *)&led_a,
tskIDLE_PRIORITY + 1, NULL);
   xTaskCreate(led_task,  "led_2s",  configMINIMAL_STACK_SIZE,  (void  *)&led_b,
tskIDLE_PRIORITY + 1, NULL);
   xTaskCreate(led_task,  "led_3s",  configMINIMAL_STACK_SIZE,  (void  *)&led_c,
tskIDLE_PRIORITY + 1, NULL);

   vTaskStartScheduler();
```

```
    for (;;) {
        /* only reached if vTaskStartScheduler() fails, e.g. out of heap */
    }
}

/* Required because FreeRTOSConfig.h sets configCHECK_FOR_STACK_OVERFLOW=2 */
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
    (void)xTask;
    (void)pcTaskName;
    for (;;) {
    }
}
```

3. Three thread priority Switch

```
#include <stdint.h>
#include "stm32f103_regs.h"

#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

#define PRIO_LOW    (tskIDLE_PRIORITY + 1)
#define PRIO_MED    (tskIDLE_PRIORITY + 2)
#define PRIO_HIGH   (tskIDLE_PRIORITY + 3)

#define BLINKS_PER_PHASE   10

typedef struct {
    GPIO_TypeDef *port;
    uint32_t      pin;
    uint32_t      period_ms;   /* full on+off blink period */
} led_task_args_t;

static const led_task_args_t led_a = { GPIOC, 13, 1000 };   /* onboard  - starts HIGH */
static const led_task_args_t led_b = { GPIOA, 0,  1000 };   /* external - stays MED   */
static const led_task_args_t led_c = { GPIOA, 1,  1000 };   /* external - starts LOW  */

static SemaphoreHandle_t xPhaseMutex;

static TaskHandle_t xHandleLed1;   /* swaps between PRIO_HIGH and PRIO_LOW */
static TaskHandle_t xHandleLed2;   /* stays at PRIO_MED, never changes      */
static TaskHandle_t xHandleLed3;   /* swaps opposite to xHandleLed1         */

static volatile uint32_t g_blinkCount = 0;   /* total blinks in this phase, all tasks combined
*/
```

```
static volatile uint32_t g_swapped = 0; /* 0: led1=HIGH,led3=LOW 1: led1=LOW,led3=HIGH */

static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
    volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
    uint32_t shift = (pin % 8U) * 4U;

    *cr &= ~(0xFU << shift);
    *cr |=  (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

/* Called once per blink, by whichever task happened to blink. */
static void maybe_swap_priorities(void)
{
    uint32_t do_swap = 0;
    uint32_t new_state = 0;

    xSemaphoreTake(xPhaseMutex, portMAX_DELAY);
    g_blinkCount++;
    if (g_blinkCount >= BLINKS_PER_PHASE) {
        g_blinkCount = 0;
        g_swapped = !g_swapped;
        do_swap = 1;
        new_state = g_swapped;
    }
    xSemaphoreGive(xPhaseMutex);

    if (!do_swap) {
        return;
    }

    /* vTaskPrioritySet() is safe to call on any task from any task -
     * it is not restricted to changing the caller's own priority. */
    if (new_state) {
        vTaskPrioritySet(xHandleLed1, PRIO_LOW);
        vTaskPrioritySet(xHandleLed3, PRIO_HIGH);
    } else {
        vTaskPrioritySet(xHandleLed1, PRIO_HIGH);
        vTaskPrioritySet(xHandleLed3, PRIO_LOW);
    }
}

static void led_task(void *pvParameters)
{
    const led_task_args_t *args = (const led_task_args_t *)pvParameters;

    for (;;) {
```

```
        args->port->ODR ^= (1U << args->pin);
        maybe_swap_priorities();
        vTaskDelay(pdMS_TO_TICKS(args->period_ms / 2));   /* half on, half off */
    }
}

int main(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPCEN;

    gpio_output_init(led_a.port, led_a.pin);
    gpio_output_init(led_b.port, led_b.pin);
    gpio_output_init(led_c.port, led_c.pin);

    xPhaseMutex = xSemaphoreCreateMutex();

    xTaskCreate(led_task, "led_1", configMINIMAL_STACK_SIZE, (void *)&led_a,
PRIO_HIGH, &xHandleLed1);
    xTaskCreate(led_task, "led_2", configMINIMAL_STACK_SIZE, (void *)&led_b,
PRIO_MED, &xHandleLed2);
    xTaskCreate(led_task, "led_3", configMINIMAL_STACK_SIZE, (void *)&led_c,
PRIO_LOW, &xHandleLed3);

    vTaskStartScheduler();

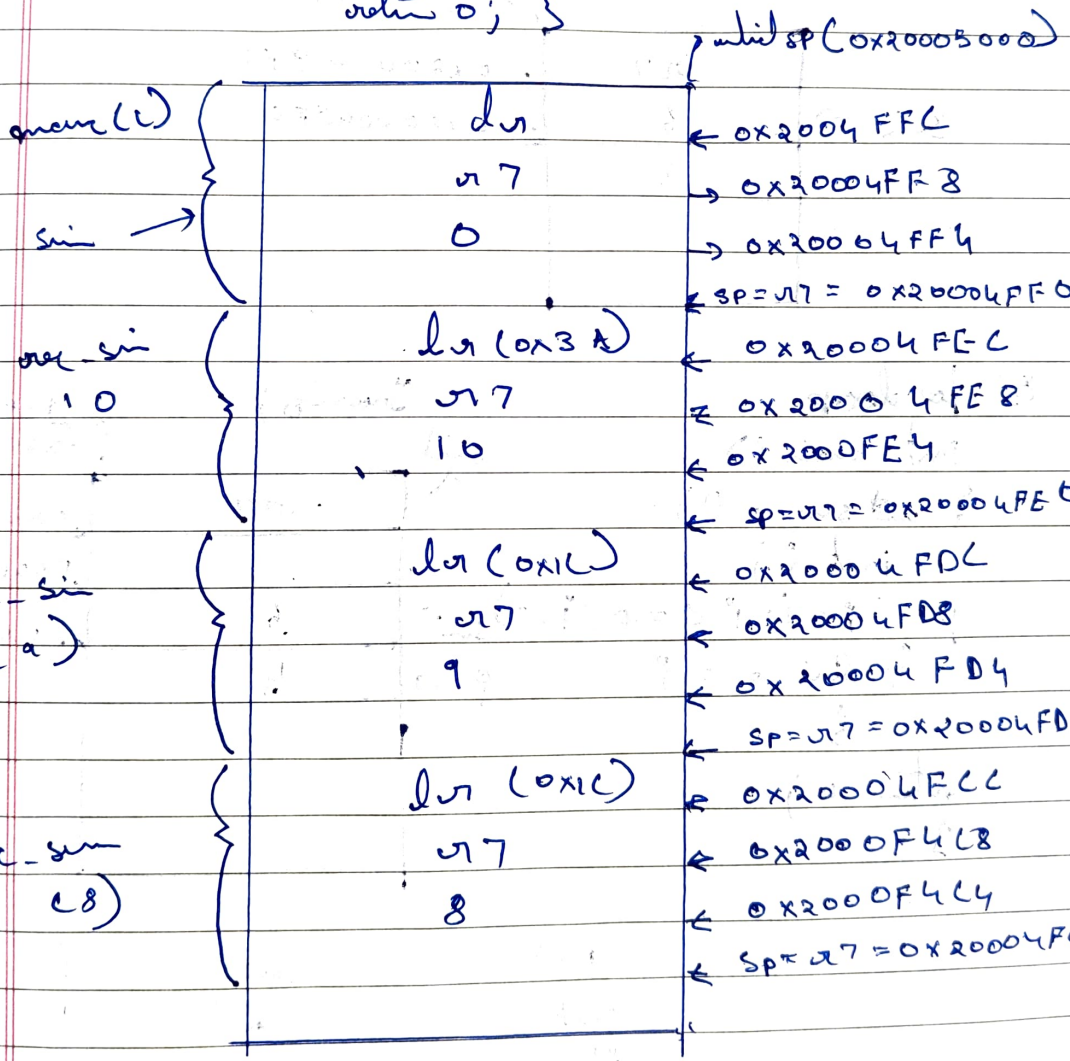
    for (;;) {
        /* only reached if vTaskStartScheduler() fails, e.g. out of heap */
    }
}

/* Required because FreeRTOSConfig.h sets
configCHECK_FOR_STACK_OVERFLOW=2 */
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
    (void)xTask;
    (void)pcTaskName;
    for (;;) {
    }
}
```

4. **For the recusrive programs, trace the gdb output and and debug the same**

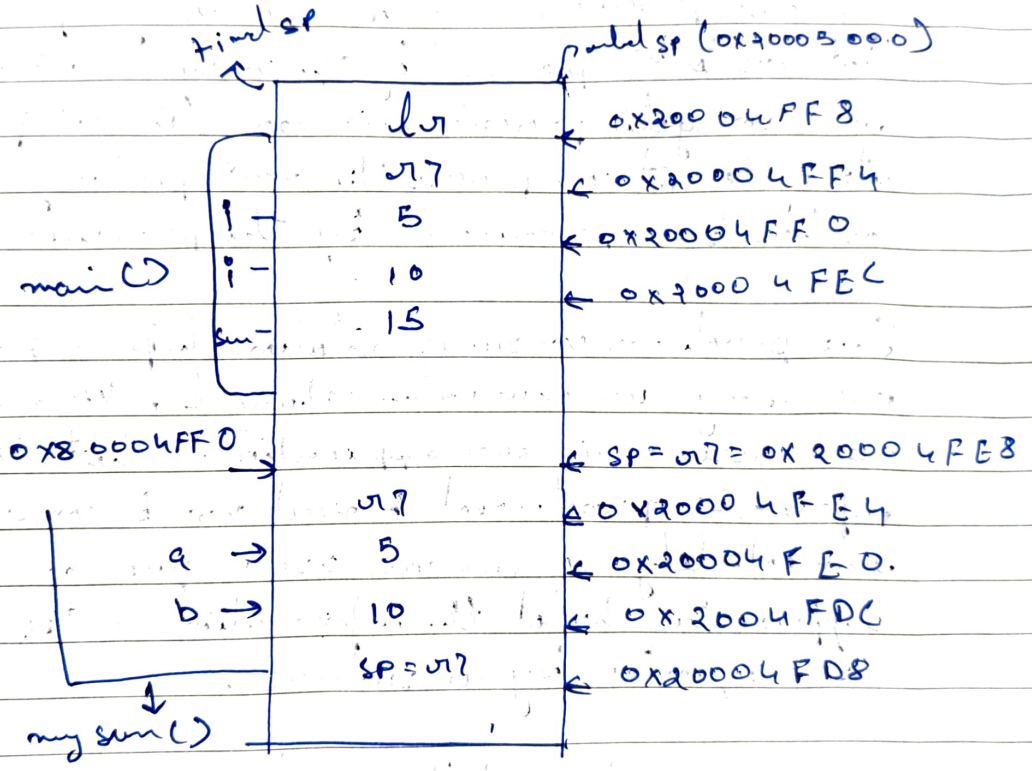
3) Recursion : int rec_sum(int i)

```
{if (i <= 1)
    return i;
 else
    return i + rec_sum(i-1) }
int main(void) {
  int sum = 0;
   sum = rec_sum(10);
   return 0; }
```



b) Just - call

```
int mysum(int a, int b)
{ return a+b; }
int main(void)
{ int p=5; i=10; sum=0;
  sum = my_sum(i, j);
  return 0;
}
```



my sin() ; -(gdb) printf %x &PC;

&PC = 0x8000040

(gdb) print /x &SP

&SP = 0x20004FE8

(gdb) print /x $lr

$lr = 0x08000075

(gdb) continue

* control come back to main2()

main() :- (gdb) print /x &PC

$PC = 0x08000074

(gdb) print /x $r0

$r0 = 15

After main() :-

(gdb) print /x $PC

$PC = 0x00000078

(gdb) print /x $SP

$SP = 0x20004FF8

* GDB Terminal

→ gdb-multiarch myfirm.elf

(gdb) target extended-remote :4243

(gdb) monitor reset halt

(gdb) break main

(gdb) continue

Breakpoint 1 main().

main() ;

(gdb) print /x $PC

$PC = 0x0800037A

classmate
Date
Page

gdb prt /x = $SP
$sp = 0x080000A9
(gdb) prt /x $SP
$sp = 0x20005000

(gdb) breach my sum
(gdb) contin
Breakpoint 2, my sum()

(gdb) prt /x $PC
$PC = 0x0800006A
(gdb) prt /x $ dir
$ dir = 0x08000081
(gdb) breach rec_sum
(gdb) contin

rec_sum(10) :-
(gdb) prt /x $PC
$PC = 0x80000040
(gdb) prt /x $ SP
$sp = 0x2000FFF0
(gdb) prt /x $r0
$ r0 = 10
(gdb) prt /x = $dir
$ dir = 0x080000.7B
(gdB) contin

rec_sum(9) :-
(gdb) prt /x $PC
$PC = 0x08000040
(gdb) prt /x $SP
$ SP = 0x2000FFE0
(gdb) prt /x $dir
dir = 0x08000050

(gdb) print /x $r0
$r0 = 9

3) Functions

(gdb) display $SP
$SP = (void *) 0x20005000
(gdb) display /x $lr
/x $lr = 0x08000139
(gdb) step:
0x08000138 in Reset_Handler ()
0x08000138 Reset_Handler

4) Recursion

C-code

```
int rec_sum (int i)
{
    if (i<=1)
        return i;
    else
        return i + rec_sum (i-1);
}

int main (void)
{
    int sum = 0
    sum = rec_sum (10);
    return 0;
}
```

* GDB commands

→ gdb multiboard recursion.elf
(gdb) target extended-remote :4242
(gdb) monitor reset halt

(gdb) break main
(gdb) continu

Breakpoint 1 main at main3.6

$sp = 0x20005000